BarrasAR: catálogo abierto de productos argentinos — Plan del proyecto

Oct 29, 2026 · @Juan

Resumen

BarrasAR (repositorio barras-ar) es un servicio de código abierto que, a partir de un código de barras (EAN), devuelve la información del producto: descripción, marca y presentación. Se alimenta solo, todos los días, con los datos públicos de SEPA (Precios Claros), que ya traen los códigos de barras de miles de productos de consumo masivo vendidos en Argentina.

Hoy esos datos existen, pero están repartidos en decenas de ZIP diarios por cadena, con descripciones distintas para el mismo producto. Nadie los ofrece como un catálogo limpio y consultable por código.

Objetivos

- **Portfolio**: un proyecto propio, publicado en GitHub, para mostrar en LinkedIn y que otros desarrolladores puedan usar.
- **Keltron**: que un comercio pase el lector (o escriba el código) al cargar una compra y el producto se complete solo, sin tipear la descripción.
- **Más adelante**: precios de referencia minoristas y mayoristas, y aportes de la comunidad.

Fuera del alcance de la primera versión: precios, historial de precios, imágenes de productos y carga colaborativa.

La fuente de datos

La fuente es la Base SEPA (Sistema Electrónico de Publicidad de Precios Argentinos), que el Estado publica todos los días como datos abiertos bajo licencia CC BY 4.0.

Dato	Minorista	Mayorista
Dataset	Precios Claros – Base SEPA	Precios Claros – SEPA Mayorista
Qué contiene	Precios de comercios minoristas (grandes establecimientos) de más de 70.000 productos, unos 12 millones de registros por día	Precios de productos de consumo masivo de comercios mayoristas con salón de ventas

Dato	Minorista	Mayorista
Responsable	Subsecretaría de Defensa del Consumidor y Lealtad Comercial; mantenedor: Dirección de Evaluación y Monitoreo de Precios	Subsecretaría de Defensa del Consumidor y Lealtad Comercial
Licencia	Creative Commons Attribution 4.0	Creative Commons Attribution 4.0
Actualización	Diaria	Diaria
Archivos	7 recursos, uno por día de la semana, cada uno un ZIP que se reemplaza semana a semana	Igual: `sepa_lunes.zip` a `sepa_domingo.zip`, más el traductor de provincias y el PDF de metadata
Precios	Precio de lista, precio de referencia y dos promociones	Con y sin IVA, por unidad y por bulto (`unidad_venta`, `precio_bulto_con_iva`), y código DUN-14 del bulto
Uso en el proyecto	Catálogo (etapa 1) y precio de referencia de venta (etapa 3)	Precio de referencia de costo (etapa 3). Usa el mismo `id_producto`, así que se cruza directo por código

El dataset minorista también incluye un Excel para traducir los códigos de provincia y un PDF de metadata con el detalle de cada campo (Anexo II de la Res. N.º 678/2020).

Contacto para consultas o datos erróneos: soportesepa@comercio.gob.ar

Estructura de los archivos

El día descargado (23/10/2026) trajo 17 ZIP, uno por cadena de comercios, y cada ZIP tiene siempre los mismos tres CSV.

- El ZIP del día es un “ZIP de ZIPs”: adentro hay una carpeta `AAAA-MM-DD/` con un ZIP por cadena.
- Nombre de cada ZIP de cadena: `sepa_<1|2>_comercio-sepa-<id_comercio>_<fecha>_<hora>.zip`. Vinieron dos tandas con distinto horario (`sepa_1_…_09-05-11` y `sepa_2_…_01-05-07`) y ninguna cadena apareció en las dos.
- Un ZIP vino vacío (`comercio-sepa-36`, 0 bytes): el proceso tiene que tolerarlo.
- Formato de los CSV: separados por `|`, UTF-8 con BOM, fin de línea CRLF. Terminan con una línea en blanco y otra de texto, por ejemplo `Ultima actualizacion: 2026-10-22T16:00:00-03:00`, que hay que descartar. Los valores con `|` o comillas van entre

comillas dobles.

Archivo	Una fila por	Columnas
`comercio.csv`	Bandera de la cadena	id_comercio, id_bandera, comercio_cuit, comercio_razon_social, comercio_bandera_nombre, comercio_bandera_url, comercio_ultima_actualizacion, comercio_version_sepa
`sucursales.csv`	Local	id_comercio, id_bandera, id_sucursal, sucursales_nombre, sucursales_tipo, sucursales_calle, sucursales_numero, sucursales_latitud, sucursales_longitud, sucursales_observaciones, sucursales_barrio, sucursales_codigo_postal, sucursales_localidad, sucursales_provincia, y el horario de atención de lunes a domingo (7 columnas)
`productos.csv`	Producto en un local	id_comercio, id_bandera, id_sucursal, id_producto, productos_ean, productos_descripcion, productos_cantidad_presentacion, productos_unidad_medida_presentacion, productos_marca, productos_precio_lista, productos_precio_referencia, productos_cantidad_referencia, productos_unidad_medida_referencia, productos_precio_unitario_promo1, productos_leyenda_promo1, productos_precio_unitario_promo2, productos_leyenda_promo2

Las columnas clave para el catálogo son `id_producto` (el código, 13 dígitos) y `productos_ean`: vale `1` si el código es un EAN real y `0` si es un código interno de la cadena.

```
id_comercio|id_bandera|id_sucursal|id_producto|productos_ean|productos_descripcion|productos_cantidad_presentacion|productos_unidad_medida_presentacion|productos_marca|productos_precio_lista
3|1|3|7791813434412|1|7UP FREE PET X 1.5L|1.00|unidad| |5200.00
3|1|3|0000000023897|0|AFEITADORA BIC COMFORT  3|1.00|unidad| |820.00
```

Lo que encontramos en los datos

Los datos sirven: un producto típico de kiosco aparece en casi todas las cadenas con su EAN, pero cada cadena lo describe distinto y con unidades distintas.

Muestra revisada: comercio 3 (DEHEZA S.A.I.C.F. e I., autoservicios 24 h)

- 43 sucursales en `sucursales.csv`, 35 con productos informados.
- 8.915 filas en `productos.csv`, pero solo 255 productos distintos (cada producto se repite una vez por sucursal).
- 211 de esos 255 tienen EAN real (`productos_ean = 1`); 44 son códigos internos, como `0000000023897` o `1111111111145`.
- La marca vino vacía en casi todos los productos de esta cadena.

Ejemplo: 7790040147720, Galletitas Surtido Bagley 400 g

Aparece en 7 de las 16 cadenas con datos ese día. Precio de lista de la primera sucursal de cada cadena:

Cadena (id)	Descripción	Presentación tal como viene	Marca	Precio de lista
2	GALLETITAS SURTIDAS, BAGLEY, 400 gr	400 · gr	BAGLEY	$3.950
10	GALLETITAS SURTIDO BAGLEY BOLSA X 400 GRS	1 · UNI	BAGLEY	$4.059 (promo $3.985)
11	BAGLEY GALLETITAS SURTIDO 400G	0.4 · KG	BAGLEY	$3.679
12	GALLETITAS DULCES SURTIDO BAGLEY PAQ 400 GRM	400 · grm	SURTIDO BAGLEY	$3.869
13	GALL SURTIDO BAGLEY	400.00 · gr	BAGLEY	$3.500
15	GALLETITAS SURTIDAS	400.00 · GR	BAGLE	$3.869
47	G.SURTIDO BAGLEY X400G.	00400 · gr	(vacía)	$2.500

Problemas que el proceso tiene que resolver

1. **Descripción**: elegir la más completa entre todas las cadenas (acá, “Galletitas Surtido Bagley 400 g” y no “GALLETITAS SURTIDAS”).

2. **Marca**: vacía, con errores (“BAGLE”) o mezclada con el producto (“SURTIDO BAGLEY”); conviene tomar la más repetida.
3. **Presentación**: unidades distintas para lo mismo (gr, GR, grm, KG, kgm, UNI, unidad, EA) y números con formatos distintos (`00400` , `0.4` , `400.00`). Hay que normalizar a g, kg, ml, l o unidades.
4. **Códigos**: descartar los que tengan `productos_ean = 0` y validar el dígito verificador de los EAN (7790040147720 lo valida).
5. **Formatos de número**: precios con ceros adelante (`02500.00`), relevante para la etapa de precios.
6. **Archivos**: tolerar ZIP vacíos y descartar la línea final de texto de cada CSV.
7. **Provincias**: no siempre vienen en ISO 3166-2. Aparecen “Buenos Aires”, vacías o sucursales sin datos (unas 33.000 de 1 millón de combinaciones código-provincia).
8. **Fecha de actualización del comercio**: no es confiable (Changómas informa 2021 y Unicoop, 2025). Hay que usar la fecha del archivo.

Un día completo: jueves 29/10/2026

Un día trae 76.581 códigos de barras válidos de 16 cadenas, y el catálogo cambia poco de una semana a otra.

Dato	Valor
Cadenas con datos	16, más un ZIP vacío (cadena 36)
Filas de productos	14,3 millones
Códigos de barras únicos válidos	76.581 (77.207 sumando el viernes 23/10)
Cambios respecto del 23/10	1.311 códigos nuevos; 626 que no estuvieron
Códigos en una sola cadena	55.466 (72%)
Códigos en 6 o más cadenas	4.166
Marcados como EAN con dígito verificador inválido	71 códigos
Filas con unidad que no se pudo normalizar	0,05%
Tiempo de procesamiento del día	2 min 40 s (Python, sin optimizar)

Cadenas: Carrefour (INC), DIA, Changómas (Dorinka), Cencosud (Vea, Disco, Jumbo), Coto, La Anónima, Cooperativa Obrera, Toledo, Mariano Max, California, Comodín, Unicoop, La Agrícola Regional, Axion Energy, Estación Lima y Deheza.

Tamaño medido en PostgreSQL

Con el modelo final, una sola tabla, el catálogo completo de un día ocupa 25 MB con el índice de búsqueda incluido (14 MB de datos), muy lejos de los 500 MB de Supabase.

Medición con el jueves 29/10/2026	Resultado
Filas de `productos`	76.581
Tamaño total con índices	25 MB
Búsqueda “surtid” + “bagley” sobre `texto_busqueda`	18 ms, las galletitas primeras
Búsqueda por código exacto	menos de 1 ms

El primer modelo, con las variantes por cadena y los precios por provincia en tablas aparte, ocupaba 289 MB. Se descartó: esos datos se recalculan cada día desde la fuente y no hace falta guardarlos.

Prueba de la regla de descripción

En 100 productos al azar vendidos por 2 o más cadenas, revisados a mano, la regla eligió una descripción correcta en 93, la presentación correcta en unos 92 y la marca correcta en unos 96.

- **Mejoras ya aplicadas**: quitar el sufijo que agrega una cadena (“BSA-80-gr.”, “BOT-750-ml.”) y elegir la presentación por separado, tomando la más repetida y descartando el “1 unidad” genérico.
- **Fallas que quedan**: descripciones abreviadas o cortadas que ganan igual (“GUADAL COL MIS PRIME”), una con el nombre de la cadena (“CUCHILLA CARREFOUR…”) y marcas abreviadas (“FERNA” por Fernando).
- **Próxima mejora**: cuando todas las cadenas informan “1 unidad”, sacar la cantidad del texto (“X 10 ML”, “X 6 UNI”).

Uso legal

Se puede: la licencia CC BY 4.0 del dataset permite copiar, transformar y redistribuir los datos, incluso con fines comerciales (Keltron), siempre que se dé crédito a la fuente.

Obligaciones

- **Atribución** en el README, en la web y en la documentación de la API. Texto sugerido: “Datos: Precios Claros – Base SEPA, Subsecretaría de Defensa del Consumidor y Lealtad Comercial. Licencia CC BY 4.0. Datos procesados y modificados por [nombre

del proyecto].”

- **Indicar que los datos fueron modificados** (limpieza, unificación, normalización).
- **No dar a entender que el Gobierno respalda el proyecto.**

Lo que dicen los términos de Precios Claros

- El Gobierno no responde por la veracidad de los datos: los informan los comercios. El proyecto debe aclarar lo mismo.
- Las marcas, imágenes y logos de productos y comercios son de sus titulares: usar el nombre del producto como dato está bien; logos e imágenes de marcas, no.
- El contenido del sitio preciosclaros.gob.ar figura bajo CC BY 2.5 Argentina; el dataset, bajo CC BY 4.0. Las dos piden atribución.

Licencias recomendadas para el proyecto

Parte	Licencia
Código	MIT
Datos que devuelve la API	CC BY 4.0, la misma que la fuente
Aportes de la comunidad (etapa 3)	CC BY 4.0, aceptada explícitamente por quien aporta

Esto no es asesoramiento legal. Si el proyecto se convierte en un servicio pago relevante, conviene una consulta con un abogado de propiedad intelectual.

Alternativas y proyectos existentes

No encontramos una API pública que devuelva productos argentinos por código de barras con buena cobertura: ese es el hueco que cubre el proyecto.

Opción	Qué ofrece	Limitación
SEPA / Precios Claros	Datos oficiales diarios con EAN	Solo descarga en ZIP; no encontramos una API pública documentada. La web usa un sistema interno que no conviene usar.

Opción	Qué ofrece	Limitación
Open Food Facts	API gratuita por código: `GET https://world.openfoodfacts.org/api/v2/product/{codigo}`; para leer no hace falta clave	Colaborativa, centrada en alimentos; cobertura argentina incompleta. Sirve como respaldo.
GS1 Argentina	Asigna los códigos; registro más completo (unos 4,8 millones de productos en 2014, según Infonegocios)	No encontramos acceso para terceros. Se puede consultar directamente.
APIs pagas (por ejemplo, Zylalabs)	Búsqueda por código	Costo y cobertura argentina sin verificar.

Proyectos en GitHub que ya usan SEPA, enfocados en precios y no en un catálogo por código:

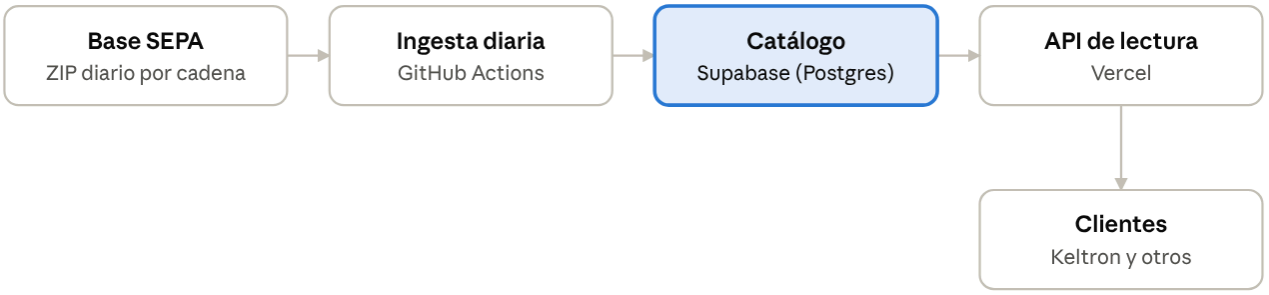
- gondolar: publica datos de SEPA procesados.
- changuito-sepa: precios de Carrefour y Coto por zona para una app.
- precios-madryn, precios-sepa-demo y precios_minoristas_supermercados: precios locales o análisis.

Vale la pena mirarlos antes de empezar: pueden ahorrar trabajo en la descarga y el parseo. El diferencial de este proyecto es un **catálogo por código de barras, limpio y con API, pensado para integrarse en sistemas de gestión**.

Arquitectura propuesta

Tres piezas: un proceso diario que baja y limpia los datos, una base con una fila por código de barras y una API de solo lectura que consulta esa base.

Un proceso diario alimenta el catálogo que consulta la API



arquitectura · fuente, ingesta, catálogo, API

La API nunca lee los ZIP: solo consulta el catálogo, que la ingesta actualiza una vez por día.

1. Proceso de ingesta (diario)

1. Bajar el ZIP del día desde el dataset y saltearlo si ya se procesó (guardar la fecha y un hash del archivo).
2. Abrir cada ZIP de cadena; saltear los vacíos o corruptos y registrar el error.
3. Leer `productos.csv` (separador `|`, UTF-8 con BOM, descartar la línea final de texto).
4. Quedarse con las filas `productos_ean = 1` cuyo código pase el dígito verificador EAN-13 (y EAN-8/UPC si aparecen).
5. Agrupar por código y por cadena (una fila por cadena, no por sucursal).
6. Normalizar la presentación (número + unidad: g, kg, ml, l, unidad).
7. Actualizar el catálogo: crear los códigos nuevos, actualizar `ultima_vez_visto`, recalcular la descripción y la marca elegidas.

2. Elección de la descripción y la marca

- **Descripción**: entre las variantes de cada cadena, preferir la que incluye la marca y la presentación y tiene más palabras completas (sin abreviaturas como “GALL” o “G.”). Antes, quitar el sufijo que agregan algunas cadenas (“BSA-80-gr.”).
- **Marca**: la más repetida entre cadenas, ignorando las vacías.
- **Presentación**: elegida por separado, la más repetida entre cadenas y normalizada a g, ml o unidad, descartando el “1 unidad” genérico.
- **Texto de búsqueda**: las palabras de todas las variantes juntas, sin repetir, en minúscula y sin tildes. Así “galletitas bagley”, “surtidas bagley” o “bagley surtido” encuentran el mismo producto aunque la descripción elegida sea abreviada.
- **Precio de referencia**: mediana, mínimo y máximo nacional del precio de lista (sin promociones), tomando la mediana de cada cadena.
- **Categoría**: SEPA no la trae; se infiere de la descripción con reglas por palabras clave.

- Las variantes de cada cadena no se guardan: se usan durante la ingesta y se descartan. Si cambia la regla, la ingesta siguiente lo recalcula todo.

3. Modelo de datos (etapa 1)

Una sola tabla con la ficha de cada producto, más un registro chico de las ingestas.

Columna de `productos`	Ejemplo	Qué es
ean	7790040147720	Código de barras, clave primaria
descripcion	GALLETITAS SURTIDO BAGLEY BOLSA X 400 GRS	La mejor descripción entre las cadenas
marca	BAGLEY	La marca más repetida
cantidad, unidad	400 · g	Presentación normalizada (g, ml o unidad)
categoria	Galletitas	Inferida por palabras clave
texto_busqueda	galletitas surtido bagley bolsa x 400 grs g x400g gall dulces paq grm surtidas 400g gr	Palabras de todas las variantes, para buscar
precio_mediana, precio_min, precio_max	3.859,50 · 2.500 · 4.059	Precio de lista nacional, sin promociones
cadenas	7	En cuántas cadenas aparece
fecha_precio, primera_vez, ultima_vez	2026-10-29	Fechas del precio y de cuándo se vio el código

`ingestas`: fecha, archivo, hash, ZIP procesados, ZIP con error, productos nuevos y actualizados. Sirve para no procesar dos veces el mismo archivo.

4. API (solo lectura)

Endpoint	Para qué
`GET /productos/{ean}`	Datos de un producto por código; 404 si no está
`GET /productos?q=galletitas bagley`	Búsqueda por texto, paginada
`GET /estadisticas`	Cantidad de productos, última ingesta, cobertura

Endpoint	Para qué
`GET /salud`	Estado del servicio

Ejemplo de respuesta de `GET /productos/7790040147720` (la descripción final depende de la regla de elección):

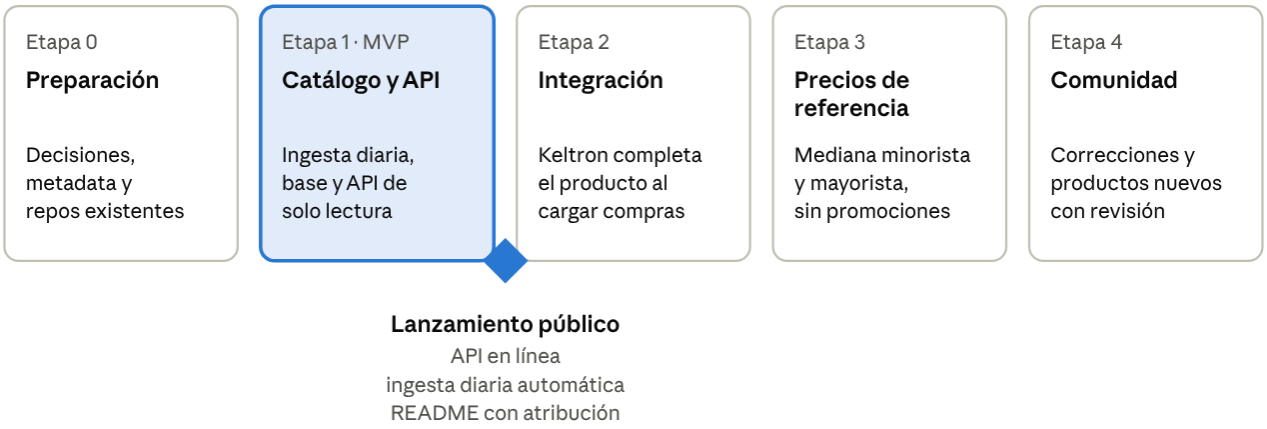

```
{
  "ean": "7790040147720",
  "descripcion": "Galletitas Surtido Bagley Bolsa x 400 g",
  "marca": "Bagley",
  "cantidad": 400,
  "unidad": "g",
  "categoria": "Galletitas",
  "precio_referencia": { "mediana": 3859.5, "min": 2500, "max": 4059,
"fecha": "2026-10-29" },
  "cadenas": 7,
  "ultima_vez": "2026-10-29",
  "fuente": "Precios Claros – Base SEPA (CC BY 4.0)"
}
```


- Con límite de pedidos por IP desde el principio, para que un servicio gratuito no se caiga por abuso.
- Atribución a la fuente en cada respuesta y en la documentación.
- Si un código no está, Keltron puede consultar Open Food Facts como respaldo (del lado de Keltron, no de la API).

Hoja de ruta

La etapa 1 es lo que se publica en GitHub y LinkedIn; las demás se suman sobre esa base, sin fechas fijas.

El MVP es el catálogo con API; precios y comunidad vienen después



hoja de ruta · 5 etapas, 1 hito

El lanzamiento público separa el MVP del resto: a partir de ahí, Keltron ya puede usar la API.

Etapa 0 · Preparación

- [ ] Lenguaje, base de datos y hosting: definidos (ver Decisiones). Nombre: BarrasAR.
- [ ] Leer el PDF de metadata y revisar dos o tres de los repositorios existentes.
- [ ] Confirmar cómo descargar automáticamente cada recurso diario.
- [ ] Crear el repositorio con README, licencias (MIT para el código, CC BY 4.0 para los datos) y el texto de atribución.

Etapa 1 · Catálogo y API (MVP)

- [ ] Lector de ZIP y CSV que tolere el BOM, el separador `|`, la línea final de texto y los ZIP vacíos.
- [ ] Filtro `productos_ean = 1` y validación del dígito verificador.
- [ ] Normalización de la presentación (cantidad y unidad).
- [ ] Regla de elección de descripción y marca, medida con una muestra revisada a mano.
- [ ] Base de datos con carga incremental diaria.
- [ ] API con los cuatro endpoints, límite de pedidos y atribución.
- [ ] Tests usando los ZIP del 23/10/2026 como datos de prueba.
- [ ] Ingesta automática diaria.
- [ ] README con ejemplos de uso y post de lanzamiento en LinkedIn.

Etapa 2 · Integración en Keltron

- [ ] Al cargar una compra o pasar el lector, consultar la API pública (como cualquier cliente) y completar descripción, marca y presentación.

- [ ] Respaldo con Open Food Facts cuando el código no está.
- [ ] Recién con esto funcionando, sumar un reel a las redes de Keltron mostrando la carga automática.

Etapa 3 · Precios de referencia

- [ ] Revisar el dataset mayorista (cobertura, formato, códigos).
- [ ] Calcular mediana, mínimo y máximo nacional del precio de lista por código, sin promociones. Por provincia, solo si hace falta más adelante.
- [ ] Mostrarlo como “precio de referencia”: minorista como venta en supermercados, mayorista como costo aproximado. No como precio sugerido, porque un kiosco suele vender más caro.
- [ ] Sumar el precio de referencia a la respuesta de `GET /productos/{ean}`.

Etapa 4 · Comunidad

- [ ] Web para buscar productos y sugerir correcciones o productos que falten.
- [ ] Revisión antes de publicar, marca de fuente (SEPA o comunidad) y protección contra spam.
- [ ] Aceptación de la licencia CC BY 4.0 por quien aporta.

Decisiones

Todo el proyecto corre en planes gratuitos: TypeScript, PostgreSQL en Supabase y la ingesta en GitHub Actions, porque Vercel no puede procesar los ZIP.

Tema	Decisión	Límite o motivo
Lenguaje	TypeScript / Node.js	Mismo stack para reutilizar en Keltron
Base de datos	PostgreSQL en Supabase, plan gratuito	500 MB por proyecto, 5 GB de egress, hasta 2 proyectos activos; el proyecto se pausa tras 1 semana sin actividad
Ingesta diaria	GitHub Actions en el repositorio público	Los runners estándar son gratis en repos públicos. En Vercel Hobby una función dura como máximo 5 minutos, y hay ZIP de más de 100 MB

Tema	Decisión	Límite o motivo
API pública y web	Vercel, plan Hobby	Gratis para proyectos personales y no comerciales, como esta API abierta
Precios (etapa 3)	Solo el último valor: mediana, mínimo y máximo nacional, dentro de la ficha del producto	Medido: la tabla completa ocupa 25 MB
Variantes por cadena	No se guardan: se usan en la ingesta para elegir la descripción y armar el texto de búsqueda	Si cambia la regla, la ingesta siguiente recalcula todo desde la fuente

Solo API, todo gratis. El proyecto es una API gratuita y abierta, sin fines comerciales, y corre solo en planes gratuitos (sin servidores pagos). Keltron la consulta como cualquier otro cliente, sin accesos especiales. No se publica una base descargable: quien quiera los datos completos los baja de la fuente oficial de SEPA.

Pendientes

- [x] ~~**Nombre**: BarrasAR, repositorio `barras-ar`.~~
- [x] ~~**Descarga automática**: el portal es CKAN y cada día de la semana es un recurso fijo con URL estable, del tipo `/dataset/<id>/resource/<id>/download/sepa_<día>.zip` (así figura en el dataset mayorista). La ingesta toma la URL del día desde la API de CKAN (`package_show` del dataset `sepa-precios`). Según changuito-sepa, el archivo del día se publica cerca de las 16:20: la ingesta corre a la noche y, si el archivo todavía no está, no toca la base.~~
- [x] ~~**`sepa_1`** y **`sepa_2`**: son dos corridas de publicación del portal, a las 09:05 y a las 13:05 (el "01-05-07" del nombre está en formato de 12 horas: los ZIP internos tienen hora 13:05). El horario fue el mismo el 23/10 y el 29/10. Cada cadena aparece en una sola corrida, pero no siempre en la misma: Cooperativa Obrera estuvo en `sepa_1` el 23/10 y en `sepa_2` el 29/10. Regla: procesar todos los ZIP del día.~~
- [x] ~~**Metadata**: leída la del dataset mayorista (Anexo II del procedimiento SEPA, Res. 12/2016). Confirma que si `productos_ean = 1` el código es EAN (GTIN-8 o GTIN-13) o UPC-A (GTIN-12), y si es `0`, un código interno de 13 dígitos. La marca es obligatoria ("Sin marca" si no tiene), aunque en la práctica viene vacía.~~
- [x] ~~**Metadata minorista** (Anexo II de la Res. 678/2020, firmado el 4/12/2020): leída. Lo que importa para el proyecto:~~
  - **Unidades permitidas**: l, hl, ml, m3, cm3, kg, gr o g, m, dm, cm, unidad y paquete. En los datos reales aparecen otras (grm, GRS, KG, kgm, UNI, EA), así que la normalización necesita una tabla de equivalencias.

  - **Granel**: si el producto se vende suelto, la cantidad de presentación es 1.
  - **Precio de lista**: precio para cualquier consumidor, sin descuentos, con cualquier medio de pago. Es el que hay que usar para el precio de referencia.
  - **Precio de referencia**: precio por unidad de medida (por litro, kilo o metro), con su cantidad (1 o 100) y unidad. Sirve para comparar presentaciones distintas.
  - **Promo 1** es de alcance general y **promo 2**, especial (tarjetas, medios de pago, jubilados): ninguna entra en el precio de referencia.
  - **La descripción debe incluir** marca, presentación, cantidad y unidad. Eso respalda la regla de elegir la descripción más completa.
  - **Tipo de sucursal**: Hipermercado (más de 15 cajas), Supermercado (4 a 15), Autoservicio (1 a 3), Tradicional (mostrador sin línea de caja) y Web. En la etapa 3 se puede filtrar por tipo para que la referencia se parezca más a un comercio chico.
  - **Horario de envío**: hasta las 06:00, con rectificación hasta las 10:00. En el mayorista: de 16:00 del día anterior a 06:00, y rectificación de 08:00 a 10:00.
- [x] ~~**Dataset mayorista**: revisado (ver La fuente de datos). Falta medir cuántos productos cubre.~~
- [x] ~~**Supabase**: pausa los proyectos gratuitos con poca actividad en 7 días, pero unas pocas consultas por día a la base alcanzan. La ingesta diaria y el uso de la API lo mantienen activo. Si se pausa, se reanuda con los datos intactos hasta un año después.~~
- [x] ~~**GitHub Actions**: en repos públicos, los workflows programados se desactivan tras 60 días sin actividad en el repositorio. Hay que hacer al menos un commit antes de cumplirse ese plazo, o reactivar el workflow a mano.~~
- [x] ~~**Tamaño de la base**: medido con un día completo. Con el modelo final, una sola tabla, ocupa 25 MB (ver Tamaño medido en PostgreSQL).~~
- [x] ~~**Precisión de la regla de descripción**: 93 de 100 descripciones correctas (ver Prueba de la regla de descripción).~~
- [ ] **Categoría**: conseguir el Anexo I de la Res. 678/2020 (categorías oficiales de SEPA), armar las reglas por palabras clave y medir qué porcentaje de productos queda clasificado.

Fuentes

- Datos Abiertos de Desarrollo Productivo: Precios Claros – Base SEPA (licencia, responsable, recursos diarios)
- Datos Abiertos de Desarrollo Productivo: Precios Claros – SEPA Mayorista y su PDF de metadata
- Precios Claros: Términos y Condiciones
- Precios SEPA, Defensa del Consumidor

- El Economista: el Gobierno habilitó SEPA (agosto de 2024)
- changuito-sepa (estructura del ZIP diario y horario de publicación)
- Documentación de la API de Open Food Facts
- Infonegocios: los datos detrás del código de barras (GS1 Argentina, 2014)
- Supabase: precios y límites del plan gratuito y pausa de proyectos
- Vercel: plan Hobby y guía de uso justo
- GitHub: facturación de GitHub Actions y desactivación de workflows
- Archivos SEPA del 23/10/2026 descargados y revisados (17 ZIP, comercio 3 descomprimido y analizado; búsqueda del código 7790040147720 en todos).